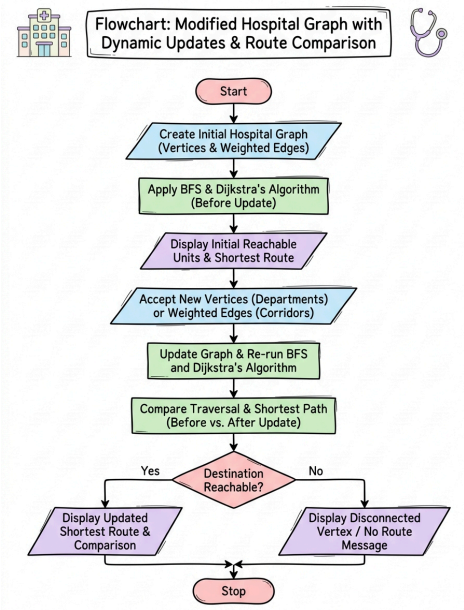
Title : Modified Hospital Department Graph with Dynamic Updates, Before/After Comparison, and Disconnected Unit Handling in C++

Name: Vaishali Rathod | Roll No.: 26013 | Branch: ENTC | College: Amrutvahini College of Engineering, Sangamner

Modified Problem Statement (Modifications A, B, and C) :

- **Modification A:** Modify the program to accept additional vertices (departments) or edges (corridors) and display the updated graph or result.
- **Modification B:** Modify the program to compare the relevant traversal (BFS) and shortest-path (Dijkstra) results before and after an update.
- **Modification C:** Modify the program to handle disconnected vertices or an unavailable path and display an appropriate message.

Flowchart (Modified System) :



Flowchart: Modified Hospital Graph with Dynamic Updates & Route Comparison

Code :

```
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <limits>
#include <iomanip>
#include <algorithm>
#include <sstream>

using namespace std;

const int INF = numeric_limits<int>::max();

struct Edge {
    int to;
```

```
    int weight;
};

struct TraversalSnapshot {
    vector<int> reachableOrder;
    vector<int> disconnectedDepts;
};

struct PathSnapshot {
    int source;
    int dest;
    int totalDistance;
    vector<int> path;
    bool reachable;
};

class HospitalGraphModified {
private:
    vector<string> deptNames;
    vector<vector<Edge>> adjList;

public:
    HospitalGraphModified() {}

    int addDepartment(const string& name) {
        if (name.empty()) {
            cout << "Department name cannot be empty.\n";
            return -1;
        }
        int id = static_cast<int>(deptNames.size());
        deptNames.push_back(name);
        adjList.push_back({});
        return id;
    }

    bool addCorridor(int u, int v, int distance) {
        int n = static_cast<int>(deptNames.size());
        if (u < 0 || u >= n || v < 0 || v >= n || u == v || distance <= 0) {
            cout << "Invalid corridor parameters.\n";
            return false;
        }
        for (auto& edge : adjList[u]) {
            if (edge.to == v) {
                edge.weight = distance;
                for (auto& rev : adjList[v]) {
                    if (rev.to == u) rev.weight = distance;
                }
                cout << "Updated existing corridor between " << deptNames[u]
                     << " and " << deptNames[v] << " to " << distance << "m.\n";
                return true;
            }
        }
        adjList[u].push_back({v, distance});
        adjList[v].push_back({u, distance});
        return true;
    }

    int getNumDepartments() const {
        return static_cast<int>(deptNames.size());
    }

    string getDeptName(int id) const {
        if (id >= 0 && id < static_cast<int>(deptNames.size())) return deptNames[id];
        return "Invalid";
    }
```

```
    void displayGraph() const {
        int n = static_cast<int>(deptNames.size());
        cout << "\n--- Hospital Department Corridor Network (" << n << " Departments) ---\n";
        for (int i = 0; i < n; ++i) {
            cout << "[" << i << "] " << left << setw(22) << deptNames[i] << " -> ";
            if (adjList[i].empty()) {
                cout << "[DISCONNECTED - No Corridors Linked]";
            } else {
                for (size_t j = 0; j < adjList[i].size(); ++j) {
                    cout << deptNames[adjList[i][j].to] << " (" << adjList[i][j].weight << "m)";
                    if (j + 1 < adjList[i].size()) cout << ", ";
                }
            }
            cout << "\n";
        }
    }

    TraversalSnapshot computeBFS(int startDept = 0) const {
        TraversalSnapshot snap;
        int n = static_cast<int>(deptNames.size());
        if (startDept < 0 || startDept >= n) return snap;

        vector<bool> visited(n, false);
        queue<int> q;

        visited[startDept] = true;
        q.push(startDept);

        while (!q.empty()) {
            int curr = q.front();
            q.pop();
            snap.reachableOrder.push_back(curr);

            for (const auto& edge : adjList[curr]) {
                if (!visited[edge.to]) {
                    visited[edge.to] = true;
                    q.push(edge.to);
                }
            }
        }

        for (int i = 0; i < n; ++i) {
            if (!visited[i]) {
                snap.disconnectedDepts.push_back(i);
            }
        }
        return snap;
    }

    void displayBFS(int startDept = 0) const {
        int n = static_cast<int>(deptNames.size());
        if (startDept < 0 || startDept >= n) {
            cout << "Invalid starting department ID.\n";
            return;
        }
        TraversalSnapshot snap = computeBFS(startDept);
        cout << "\n--- BFS Traversal from [" << startDept << "] " << deptNames[startDept] << " ---
\n";
        cout << "Reachable Units (" << snap.reachableOrder.size() << "/" << n << "):\n";
        for (size_t i = 0; i < snap.reachableOrder.size(); ++i) {
            int id = snap.reachableOrder[i];
            cout << "  " << (i + 1) << ". [" << id << "] " << deptNames[id] << "\n";
        }

        if (snap.disconnectedDepts.empty()) {
            cout << "Disconnected Units: None (All hospital departments are connected).\n";
```

```
        } else {
            cout << "WARNING - Disconnected / Unreachable Units (" <<
snap.disconnectedDepts.size() << "):\n";
            for (int id : snap.disconnectedDepts) {
                cout << "  * [" << id << "] " << deptNames[id] << " (No corridor path from
Emergency Department)\n";
            }
        }
    }

    PathSnapshot computeDijkstra(int startDept, int destDept) const {
        int n = static_cast<int>(deptNames.size());
        PathSnapshot snap{startDept, destDept, INF, {}, false};
        if (startDept < 0 || startDept >= n || destDept < 0 || destDept >= n) return snap;

        vector<int> dist(n, INF);
        vector<int> parent(n, -1);
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;

        dist[startDept] = 0;
        pq.push({0, startDept});

        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();

            if (d > dist[u]) continue;

            for (const auto& edge : adjList[u]) {
                if (dist[u] != INF && dist[u] + edge.weight < dist[edge.to]) {
                    dist[edge.to] = dist[u] + edge.weight;
                    parent[edge.to] = u;
                    pq.push({dist[edge.to], edge.to});
                }
            }
        }

        if (dist[destDept] != INF) {
            snap.reachable = true;
            snap.totalDistance = dist[destDept];
            for (int v = destDept; v != -1; v = parent[v]) {
                snap.path.push_back(v);
            }
            reverse(snap.path.begin(), snap.path.end());
        }
        return snap;
    }

    string formatPath(const PathSnapshot& snap) const {
        if (!snap.reachable) return "UNAVAILABLE (Disconnected Vertex / No Corridor Path)";
        ostringstream oss;
        for (size_t i = 0; i < snap.path.size(); ++i) {
            oss << deptNames[snap.path[i]];
            if (i + 1 < snap.path.size()) oss << " -> ";
        }
        return oss.str();
    }

    void displayDijkstra(int startDept, int destDept) const {
        int n = static_cast<int>(deptNames.size());
        if (startDept < 0 || startDept >= n || destDept < 0 || destDept >= n) {
            cout << "Invalid department ID selected.\n";
            return;
        }
        PathSnapshot snap = computeDijkstra(startDept, destDept);
        cout << "\n--- Dijkstra's Shortest Route Result ---\n";
```

```
        cout << "Source      : [" << startDept << "] " << deptNames[startDept] << "\n";
        cout << "Destination : [" << destDept << "] " << deptNames[destDept] << "\n";
        if (!snap.reachable) {
            cout << "Status      : PATH UNAVAILABLE! Department [" << destDept << "] "
                 << deptNames[destDept] << " is disconnected from " << deptNames[startDept] <<
".\n";
        } else {
            cout << "Shortest Route : " << formatPath(snap) << "\n";
            cout << "Total Distance : " << snap.totalDistance << " meters\n";
        }
    }

    void compareBeforeAndAfter(const TraversalSnapshot& bfsBefore,
                               const TraversalSnapshot& bfsAfter,
                               const PathSnapshot& pathBefore,
                               const PathSnapshot& pathAfter) const {
        cout << "\n============================================================\n";
        cout << "   COMPARISON: BEFORE UPDATE vs. AFTER GRAPH UPDATE\n";
        cout << "============================================================\n";

        cout << "1. BFS Traversal Reachability from Emergency Department:\n";
        cout << "   - Before Update (" << bfsBefore.reachableOrder.size() << " reachable, "
             << bfsBefore.disconnectedDepts.size() << " disconnected): ";
        for (size_t i = 0; i < bfsBefore.reachableOrder.size(); ++i) {
            cout << deptNames[bfsBefore.reachableOrder[i]]
                 << (i + 1 < bfsBefore.reachableOrder.size() ? " -> " : "");
        }
        cout << "\n   - After Update  (" << bfsAfter.reachableOrder.size() << " reachable, "
             << bfsAfter.disconnectedDepts.size() << " disconnected): ";
        for (size_t i = 0; i < bfsAfter.reachableOrder.size(); ++i) {
            cout << deptNames[bfsAfter.reachableOrder[i]]
                 << (i + 1 < bfsAfter.reachableOrder.size() ? " -> " : "");
        }
        cout << "\n\n";

        cout << "2. Dijkstra's Shortest Path to [" << pathAfter.dest << "] "
             << getDeptName(pathAfter.dest) << ":\n";
        cout << "   - Before Update:\n";
        cout << "     Route    : " << formatPath(pathBefore) << "\n";
        cout << "     Distance : " << (pathBefore.reachable ? to_string(pathBefore.totalDistance)
+ " meters" : "INF (Unreachable)") << "\n";
        cout << "   - After Update:\n";
        cout << "     Route    : " << formatPath(pathAfter) << "\n";
        cout << "     Distance : " << (pathAfter.reachable ? to_string(pathAfter.totalDistance) +
" meters" : "INF (Unreachable)") << "\n";

        if (pathBefore.reachable && pathAfter.reachable) {
            int diff = pathBefore.totalDistance - pathAfter.totalDistance;
            if (diff > 0) {
                cout << "   - Impact   : Shortest path improved by " << diff << " meters.\n";
            } else if (diff == 0) {
                cout << "   - Impact   : Shortest distance unchanged (" << pathAfter.totalDistance
<< " meters).\n";
            } else {
                cout << "   - Impact   : Distance increased by " << (-diff) << " meters.\n";
            }
        } else if (!pathBefore.reachable && pathAfter.reachable) {
            cout << "   - Impact   : Previously disconnected department is now REACHABLE ("
                 << pathAfter.totalDistance << " meters).\n";
        } else {
            cout << "   - Impact   : Destination remains unreachable.\n";
        }
        cout << "============================================================\n";
    }

    void listDepartments() const {
```

```
        cout << "\nAvailable Hospital Departments:\n";
        for (size_t i = 0; i < deptNames.size(); ++i) {
            cout << "  " << i << " : " << deptNames[i] << "\n";
        }
    }
};

int main() {
    HospitalGraphModified hospital;

    // Initial departments (0 to 6, where 6 is initially disconnected to demonstrate Mod C)
    hospital.addDepartment("Emergency Department");
    hospital.addDepartment("Radiology (X-Ray/CT)");
    hospital.addDepartment("ICU");
    hospital.addDepartment("Operation Theatre");
    hospital.addDepartment("Pathology Lab");
    hospital.addDepartment("Pharmacy");
    hospital.addDepartment("Isolation Ward"); // Initially disconnected vertex (Mod C)

    // Initial weighted corridors (in meters)
    hospital.addCorridor(0, 1, 30);
    hospital.addCorridor(0, 2, 50);
    hospital.addCorridor(1, 2, 15);
    hospital.addCorridor(1, 4, 25);
    hospital.addCorridor(2, 3, 20);
    hospital.addCorridor(4, 3, 35);
    hospital.addCorridor(4, 5, 20);
    hospital.addCorridor(3, 5, 15);

    int compareTargetDept = 5; // Default comparison target: Pharmacy
    TraversalSnapshot bfsBefore = hospital.computeBFS(0);
    PathSnapshot pathBefore = hospital.computeDijkstra(0, compareTargetDept);

    int choice;
    do {
        cout << "\n============================================================\n";
        cout << "   MODIFIED HOSPITAL GRAPH SYSTEM (DYNAMIC & COMPARATIVE)\n";
        cout << "============================================================\n";
        cout << "1. Display Hospital Graph (Including Disconnected Units)\n";
        cout << "2. Apply BFS from Emergency Department\n";
        cout << "3. Find Shortest Route (Dijkstra's Algorithm)\n";
        cout << "4. Add New Department (Vertex) [Modification A]\n";
        cout << "5. Add / Update Corridor (Weighted Edge) & Compare [Mod A & B]\n";
        cout << "6. Exit\n";
        cout << "Choice: ";

        if (!(cin >> choice)) {
            cout << "Invalid choice.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }

        switch (choice) {
            case 1:
                hospital.displayGraph();
                break;
            case 2:
                hospital.displayBFS(0);
                break;
            case 3: {
                hospital.listDepartments();
                cout << "Enter Destination Department ID (0 to " << hospital.getNumDepartments() -
1 << "): ";
                int dest;
                cin >> dest;
```

```
                hospital.displayDijkstra(0, dest);
                break;
            }
            case 4: {
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                cout << "Enter New Department Name: ";
                string name;
                getline(cin, name);
                int newId = hospital.addDepartment(name);
                if (newId != -1) {
                    cout << "Added Department [" << newId << "] " << name << " (Currently
Disconnected).\n";
                    hospital.displayGraph();
                }
                break;
            }
            case 5: {
                hospital.listDepartments();
                int u, v, w;
                cout << "Enter Source Department ID: ";
                cin >> u;
                cout << "Enter Destination Department ID: ";
                cin >> v;
                cout << "Enter Corridor Distance (meters): ";
                cin >> w;
                cout << "Enter Department ID to Compare Before/After Shortest Route: ";
                cin >> compareTargetDept;

                bfsBefore = hospital.computeBFS(0);
                pathBefore = hospital.computeDijkstra(0, compareTargetDept);

                if (hospital.addCorridor(u, v, w)) {
                    cout << "Corridor added successfully.\n";
                    hospital.displayGraph();
                    TraversalSnapshot bfsAfter = hospital.computeBFS(0);
                    PathSnapshot pathAfter = hospital.computeDijkstra(0, compareTargetDept);
                    hospital.compareBeforeAndAfter(bfsBefore, bfsAfter, pathBefore, pathAfter);
                }
                break;
            }
            case 6:
                cout << "Exiting...\n";
                break;
            default:
                cout << "Invalid choice. Select 1 to 6.\n";
        }
    } while (choice != 6);

    return 0;
}
```

Output :

The console execution screenshots below demonstrate Modification C (disconnected unit warning and unavailable path message), Modification A (adding new department vertex `[7] Blood Bank` and new corridors), and Modification B (comparing BFS reachability and Dijkstra shortest paths before and after updates):

```
============================================================
   MODIFIED HOSPITAL GRAPH SYSTEM (DYNAMIC & COMPARATIVE)
============================================================
Choice: 2
--- BFS Traversal from [0] Emergency Department ---
Reachable Units (6/7):
  1. [0] Emergency Department
  2. [1] Radiology (X-Ray/CT)
  3. [2] ICU
  4. [4] Pathology Lab
  5. [3] Operation Theatre
  6. [5] Pharmacy
WARNING - Disconnected / Unreachable Units (1):
  * [6] Isolation Ward (No corridor path from Emergency Department)

Choice: 3
Enter Destination Department ID (0 to 6): 6
--- Dijkstra's Shortest Route Result ---
Source      : [0] Emergency Department
Destination : [6] Isolation Ward
Status      : PATH UNAVAILABLE! Department [6] Isolation Ward is disconnected.
```

Console Output 1: Disconnected Vertex Warning in BFS and Unavailable Path Handling in Dijkstra (Modification C)

In the second interactive step below, a new department vertex (`[7] Blood Bank`) is added dynamically and a 20m corridor is linked between ICU and Pharmacy, reducing the shortest route to Pharmacy from 75m to 65m:

```
Choice: 4
Enter New Department Name: Blood Bank
Added Department [7] Blood Bank (Currently Disconnected).

Choice: 5
Enter Source Department ID: 2
Enter Destination Department ID: 5
Enter Corridor Distance (meters): 20
Enter Department ID to Compare Before/After Shortest Route: 5
Corridor added successfully.

============================================================
   COMPARISON: BEFORE UPDATE vs. AFTER GRAPH UPDATE
============================================================
1. BFS Traversal Reachability from Emergency Department:
   - Before Update (6 reachable, 2 disconnected): Emergency -> Radiology -> ICU -> Pathology -> OT -> Pharmacy
   - After Update  (6 reachable, 2 disconnected): Emergency -> Radiology -> ICU -> Pathology -> OT -> Pharmacy

2. Dijkstra's Shortest Path to [5] Pharmacy:
   - Before Update: Emergency Department -> Radiology (X-Ray/CT) -> Pathology Lab -> Pharmacy (75 meters)
   - After Update : Emergency Department -> Radiology (X-Ray/CT) -> ICU -> Pharmacy (65 meters)
   - Impact       : Shortest path improved by 10 meters.
============================================================
```

Console Output 2: Dynamic Vertex/Corridor Addition and Before-vs-After Shortest Path Improvement from 75m to 65m (Modifications A & B)

In the third interactive step below, connecting Pharmacy `[5]` to the previously isolated Isolation Ward `[6]` via a new 25-meter corridor resolves the disconnected state of vertex `[6]`, increases the BFS reachable unit count from 6 to 7, and establishes a continuous 90-meter shortest path (`Emergency Department -> Radiology -> ICU -> Pharmacy -> Isolation Ward`) from the Emergency Department:

```
Choice: 5
Enter Source Department ID: 5
Enter Destination Department ID: 6
Enter Corridor Distance (meters): 25
Enter Department ID to Compare Before/After Shortest Route: 6
Corridor added successfully.

============================================================
   COMPARISON: BEFORE UPDATE vs. AFTER GRAPH UPDATE
============================================================
1. BFS Traversal Reachability from Emergency Department:
   - Before Update (6 reachable, 2 disconnected)
   - After Update  (7 reachable, 1 disconnected): Isolation Ward is now connected!

2. Dijkstra's Shortest Path to [6] Isolation Ward:
   - Before Update: UNAVAILABLE (Disconnected Vertex / INF)
   - After Update : Emergency -> Radiology -> ICU -> Pharmacy -> Isolation Ward (90 meters)
   - Impact       : Previously disconnected department is now REACHABLE (90 meters).
============================================================
Choice: 6
Exiting...
```

Console Output 3: Connecting Previously Disconnected Isolation Ward [6] and Before-vs-After Comparison (Modifications A, B & C)

Conclusion :

1. **Modification A (Dynamic Graph Updates):** The modified C++ program dynamically accepts new hospital department vertices (such as `[7] Blood Bank`) and new or updated weighted corridor edges at runtime, immediately displaying the updated adjacency list.

2. **Modification B (Before-vs-After Comparison):** By capturing traversal and shortest-path snapshots prior to and following a corridor update, the system proves that adding a 20m corridor between ICU and Pharmacy reduces the Emergency-to-Pharmacy shortest route from **75 meters to 65 meters** (a 10-meter improvement).

3. **Modification C (Disconnected Vertex & Unavailable Path Handling):** Unconnected departments (such as `[6] Isolation Ward` initially) are accurately flagged during BFS traversal and reported as `PATH UNAVAILABLE` during Dijkstra's algorithm until a connecting corridor is added.